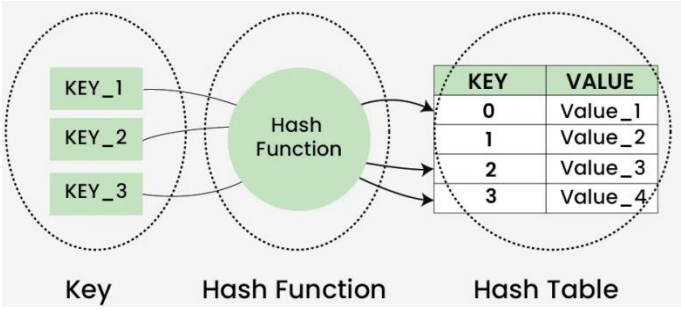
Universidade Federal de Santa Maria
Departamento de Computação Aplicada
Disciplina: Computação Gráfica Avançada – ELC 1051
Prof. Cesar Tadeu Pozzer
20/10/2025

Hash Table

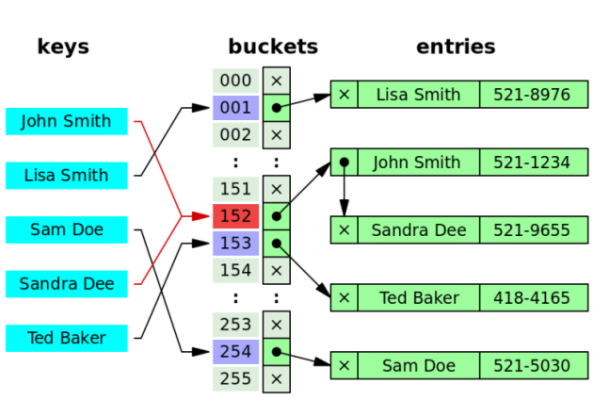
Armazena valores fazendo uso de uma **hash function**, logo os valores não são armazenados de forma ordenada como ocorre em um Map. A chave pode ser um número, uma coordenada, uma string, etc, e a **função de hash** deve ser específica para cada tipo de dado e problema a ser tratado, e deve retornar um valor inteiro.

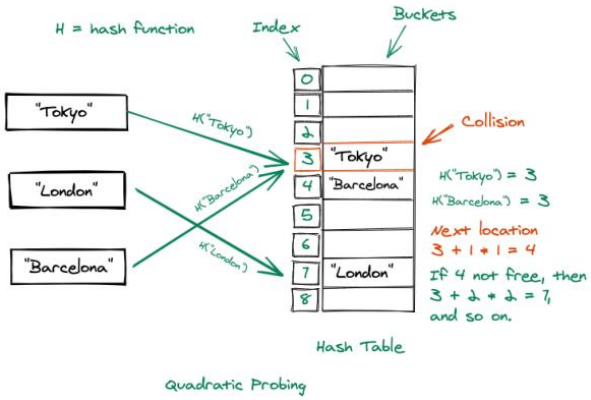
```
index = hashFunction(key)
Arr[index] = (key, values)
```



Caso não ocorram colisões, o tempo de acesso é O(1) tanto para leitura como escrita. Caso ocorram colisões, deve-se utilizar alguma das estratégias:

1. Lista encadeada: armazena todos os elementos que possuem a mesma chave;
2. Segunda função de hash: index = hash(key) + rehash(key)
3. Próxima posição livre no array.





Existem diversos algoritmos para definição de hash functions de strings, e muitos outros podem ser criados: https://stackoverflow.com/questions/7666509/hash-function-for-string

Hash(S) = (S[0] * P^0 + S[1] * P^1 +S[2] * P^2 + …) % M

Where,

S is the given string.

P is the prime number >= to the number of distinct characters in the string S.

M should be large to avoid the collision.

O seguinte algoritmo reduz a chance de colisão fazendo o %M a cada passo. Quanto maior o M, menor a chance de colisão.

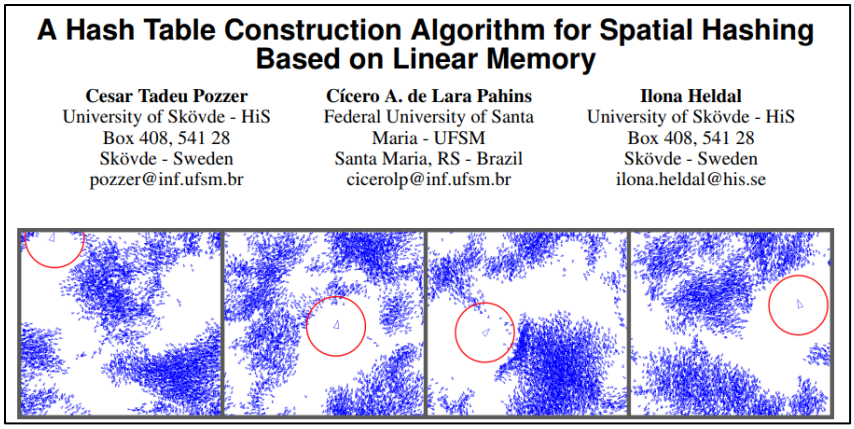
```
// Function to find the hash of string.
int hashing(string s)
{
    // To store the hash value.
    int ans = 0;

    // To store 'P'.
    int p = 1;

    // For taking modulo.
    int m = 1000000007;
    for (int i = 0; i < s.size(); i++)
    {
        ans += (s[i] - 'a') * p;
        ans = ans % m;
        p *= 31;
    }
    return ans;
}
```

Existem diversos outros materiais de hash para strings para uso em Competitive Programming: https://cp-algorithms.com/string/string-hashing.html#search-for-duplicate-strings-in-an-array-of-strings

Para hash de pontos no espaço 2d, ver artigo: **A Hash Table Construction Algorithm for Spatial Hashing Based on Linear Memory (Pozzer et al., 2014)**. Arquivo: cga_33_Hash_Espacial_Artigo.pdf. Observem que nesta solução não existem colisões.



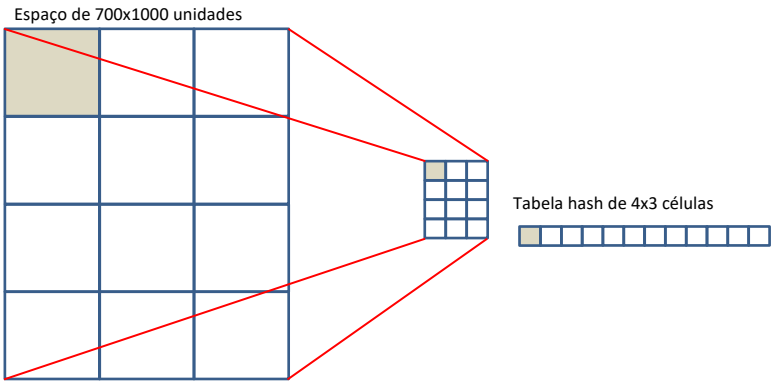
A Hash Table Construction Algorithm for Spatial Hashing Based on Linear Memory

Cesar Tadeu Pozzer
University of Skövde - HiS
Box 408, 541 28
Skövde - Sweden
pozzer@inf.ufsm.br

Cícero A. de Lara Pahins
Federal University of Santa Maria - UFSM
Santa Maria, RS - Brazil
cicerolp@inf.ufsm.br

Ilona Heldal
University of Skövde - HiS
Box 408, 541 28
Skövde - Sweden
ilona.heldal@his.se

A ideia básica do algoritmo é mapear uma área retangular 2D em uma tabela hash 2D de mxm posições, que é implementada como um array sequencial.




```
#define QUANT_ELEMENTOS 1000

#define LARGURA_TERRENO  700  //terrain size
#define ALTURA_TERRENO   1000  //terrain size

#define LARGURA_HASH   3
#define ALTURA_HASH    4
#define TAMANHO_HASH   LARGURA_HASH * ALTURA_HASH

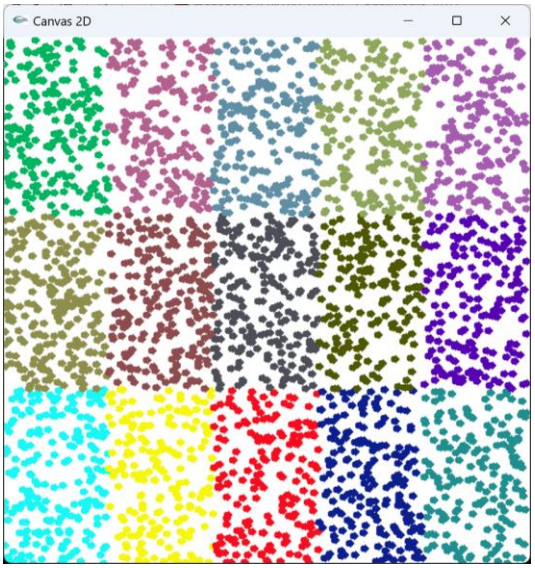
inline int hashFunction(int id)
{
    int escalaX = LARGURA_TERRENO / LARGURA_HASH;
    int escalaY = ALTURA_TERRENO / ALTURA_HASH;

    int x = elementos[id]->m_pos.x / escalaX;
    int y = elementos[id]->m_pos.y / escalaY;

    int h = y*LARGURA_HASH + x; //transformação matricial para vetorial

    return h;
}
```

A seguinte imagem representa os objetos armazenados em células específicas da tabela hash formada por 3x5 células.



O uso de tabela hash se faz necessário quando se precisa realizar muitas operações sobre o mesmo conjunto de dados. Como no exemplo do artigo, determinar a proximidade de cada entidade com cada outra, considerando que exista um raio de ação e que somente os elementos próximos devem ser processados.

Outro bom exemplo é na remoção de árvores que estão plantadas sobre estradas formadas por segmentos de reta. Deve-se verificar somente as árvores que estão próximas de cada segmento de reta. O uso de tabela hash neste caso melhora a eficiência em milhares de vezes.

Lista de exercícios sobre Hash:

1. Implemente o algoritmo proposto no artigo para obter uma imagem semelhante a que é mostrada acima. Sobre a tabela, aplique algum processamento e compare o tempo com e sem o uso da tabela hash, para diferentes números de células da tabela: 10x10, 30x30, etc, e também para diferentes quantidades de objetos: 1.000, 100.000, 10.000.000, etc.
    a. Encontrar os dois elementos mais próximos entre si;
    b. Eliminar elementos que estão muito próximos um do outro, para simular uma plantação de árvores.
2. Compare a mesma solução sem usar tabela hash. Faça uso de uma solução de força bruta.